MÉTRICAS Y ESTÁNDARES DE CALIDAD

GYM TRACK: AUDITORÍA DE CALIDAD, MANTENIBILIDAD Y EVOLUCIÓN DEL SOFTWARE

PRESENTA:

BERNAL GORDILLO JUAN CAMILO

MUÑOZ MONTENEGRO CRISTIAN ANDRÉS

QUINTERO POVEDA LAURA MARCELA

DOCENTE:

JOSÉ MIGUEL LLANOS

NEIVA – HUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA- CORHUILA

NOVIEMBRE DEL 2026

INTRODUCCIÓN

Gym Track es una aplicación web pensada para llevar el control de los estudiantes de un gimnasio y de sus sesiones de entrenamiento. Tiene dos tipos de usuario. El administrador maneja las cuentas y hace las operaciones que solo su rol permite. El entrenador, por su parte, registra y consulta a los estudiantes que tiene a cargo, abre sus sesiones, anota los grupos musculares que trabajaron, las cierra y revisa el historial.

No partimos de cero: la aplicación viene de Desarrollo Fullstack y la reutilizamos para esta materia. Lo que hicimos fue reunirla en un repositorio común, ampliar las pruebas, medir su mantenibilidad con distintas métricas, registrar cuánto esfuerzo cuesta mantenerla y evaluar tres refactorizaciones. Con todo eso quisimos responder tres preguntas: qué aspectos del sistema tienen evidencia a favor, cuáles necesitan atención y qué criterios todavía no hemos podido comprobar.

El análisis incluyó el código Java de la API, el código TypeScript del portal dentro del alcance que definimos y el esfuerzo de mantenimiento documentado en el repositorio. La línea base se midió en el commit f0669d6 y cada refactorización se midió en su propia rama. Para comprobar que todo funcionaba en conjunto, verificamos la integración en el commit fa80144.

REQUISITOS Y TRAZABILIDAD

Esta tabla muestra qué debe hacer la aplicación en cada requisito funcional, cómo sabemos que se cumple y dónde está la evidencia.

Requisitos Funcionales

ID	Descripción	Criterio de aceptación	Componente (trazabilidad)
RF-01	El usuario puede iniciar sesión.	Con credenciales correctas recibe un token JWT y su perfil; con credenciales incorrectas recibe un error 401.	AuthController y AuthService; pruebas de autenticación y los logins de la colección de Postman.
RF-02	El administrador puede registrar y consultar usuarios.	Un alta correcta devuelve 201; no se acepta un nombre de usuario repetido; el entrenador no puede usar este recurso.	UserControllerTest y UserServiceTest.
RF-03	El administrador puede modificar y eliminar usuarios.	Si no envía contraseña, se mantiene la actual; no se aceptan nombres repetidos; no puede borrar su propia cuenta ni la de un entrenador con estudiantes asignados.	UserControllerTest y UserServiceTest. La revisión detectó un permiso excesivo en el cambio de contraseña.
RF-04	El entrenador puede registrar estudiantes.	El alta devuelve 201 y el estudiante queda asignado a quien lo creó; si el documento ya existe, devuelve 409.	StudentControllerTest y StudentServiceTest.
RF-05	Cada rol consulta lo que le corresponde.	El entrenador ve solo sus estudiantes y recibe 404 si pide uno ajeno; el administrador puede ver los de todos los entrenadores.	StudentServiceTest y pruebas HTTP de consulta.
RF-06	Los estudiantes se pueden actualizar y eliminar según el rol.	El entrenador actualiza a los suyos; eliminar es cosa del administrador y se rechaza con 409 si el estudiante ya tiene sesiones.	StudentControllerTest y StudentServiceTest.

RF-07	Se puede abrir una sesión para un estudiante.	Si todo está bien devuelve 201; si ya tenía una sesión abierta, la segunda solicitud devuelve 409.	SessionServiceTest y peticiones 03 y 04 de Postman.
RF-08	Mientras la sesión está abierta se pueden registrar grupos musculares.	Repetir un grupo no lo duplica; se rechazan los grupos que no existen y cualquier cambio sobre una sesión ya cerrada.	SessionServiceTest y peticiones 06 a 08 y 11 de Postman.
RF-09	Una sesión se cierra solo si tiene al menos un grupo registrado.	No se puede cerrar sin grupos ni cerrar dos veces la misma sesión; la duración se guarda en minutos enteros.	SessionServiceTest y peticiones 05, 09 y 10 de Postman.
RF-10	Se puede consultar el historial filtrando por estado y eliminar sesiones según el rol.	Si el administrador elimina una, devuelve 204; si lo intenta el entrenador, recibe 403.	SessionControllerTest, SessionServiceTest y petición 12 de Postman (historial).

Requisitos no funcionales

ID	Categoría	Descripción medible	Forma de verificación
RNF-01	Rendimiento	P95 menor a 500 ms en cada endpoint de escritura con 20 usuarios al tiempo; listas de estudiantes, usuarios e historial se muestran en menos de 3 segundos.	Verificación parcial. Prueba local en e780adf interrumpida por límite de recursos. Falta completar los endpoints de escritura y medir la presentación de las tres listas.
RNF-02	Seguridad	Los endpoints protegidos responden 401 si no hay un token válido y 403 si el rol no tiene permiso, según el contrato.	Pruebas de acceso y de roles, y matriz completa de rutas, roles y tokens inválidos. Estado: parcialmente verificado; falta la matriz completa y se confirmó un

			permiso excesivo en la actualización de contraseñas.
RNF-03	Mantenibilidad	Ningún método de producción dentro del alcance medido supera V(G) = 10.	Medición de complejidad ciclomática sobre el alcance definido. Estado: la línea base cumple (máximo observado: exactamente 10); falta medir de nuevo sobre el commit final para afirmarlo de la versión entregada.

ARQUITECTURA

Gym Track está hecha con una arquitectura en capas, primero tenemos el portal Angular que es el que muestra las pantallas y envía las solicitudes HTTP a la API REST de Spring Boot. En la API, los controladores reciben las peticiones y utilizan dto para representar los datos de las entradas y salidas. Los servicios aplican las reglas de negocio y usan la información mediante los repositorios para consultar o guardar información en PostgreSQL.

Figura 1: Recorrido de una solicitud en Gym Track



Elegimos esta arquitectura porque es más fácil relacionar los usuarios, los estudiantes y las sesiones, un ejemplo claro de esto es que para abrir una sesión necesitamos saber quién es el entrenador y que estudiante tiene asignado. Al dividir el sistema en capas, podemos ubicar más fácil donde esta cada regla y probarla por separado. Una desventaja es que si cambiamos los datos que pide una operación, puede tocar ajustar tanto el portal como la API.

Los permisos se revisan en dos partes. En la API, Spring Security comprueba que el usuario haya iniciado sesión y que su rol le permita realizar la operación. Después el servicio revisa si ese usuario puede actuar sobre el registro especifico.

Cuando un entrenador pulsa abrir sesión, Angular envía la solicitud a la API. El controlar la pasa al servicio de sesiones, que verifica si el estudiante está asignado a ese entrenar y si ya tiene la sesión abierta. Si no hay sesiones abiertas se crea una y se guarda en PostgrSQL. Si el entrenador intenta abrir otra para el mismo estudiante, el sistema rechaza la solicitud.

IMPLEMENTACIÓN DE PRUEBAS

Gym Track tiene dos partes principales, la del portal y la de la API. El portal se hizo para que con Angular y TypeScript mostraran una parte visual que la usan el administrador y el entrenador. La API se creó con Java con Spring Boot y se encarga de procesar las solicitudes. Los datos de usuarios, estudiantes y sesiones que se guardan en PostgreSQL.

Si hablamos de módulos, nosotros tenemos una organización en cuatro de estos. Autenticación permite iniciar sesión. Usuarios que permite al administrador gestionar las cuentas. Estudiantes que registra, consulta y actualiza la información según los permisos de cada usuario. Sesiones permite iniciar un entrenamiento, registrar los grupos musculares trabajados, cerrar sesión y consultar el historial.

Para ejecutar la aplicación, debemos configurar el archivo .env siguiendo el README de src/g-t-infra. Desde esa carpeta ejecutamos docker compose up --build. Luego abrimos el portal en http://localhost:4200 y comprobamos que podíamos iniciar sesión.

Resultados

Las siguientes pruebas funcionales se ejecutaron sobre fa80144. La prueba de rendimiento posterior se reporta por separado sobre e780adf.

¿Qué probamos?	Resultado	¿Qué se comprobó?
API con Maven	95 pruebas aprobadas, sin fallos ni errores.	Casos de inicio de sesión, usuarios, estudiantes y sesiones.
Portal Angular	Compilación exitosa y 1 prueba aprobada.	La prueba verificó el guard de autenticación, que controla el acceso a las pantallas protegidas.

Arranque con Docker	La aplicación arrancó y se pudo iniciar sesión.	El portal, la API y la base de datos funcionaron juntos para el inicio de sesión.
Colección de Postman	12 peticiones, 24 comprobaciones aprobadas y 0 fallos.	Apertura de una sesión, rechazo de una apertura duplicada, registro de grupos musculares, cierre e historial.

En e780adf se ejecutó una prueba local de Postman con un perfil fijo de 20 usuarios virtuales y cinco minutos previstos. Postman detuvo la corrida por un límite de recursos del equipo. Se observaron picos de 99,9 % de CPU y 95,9 % de memoria. Hasta la interrupción se enviaron 7.900 solicitudes y el p95 global fue de 154 ms. Los p95 de las solicitudes de escritura incluidas estuvieron entre 48 y 92 ms, por debajo de 500 ms. El alta de estudiantes mostró un p95 de 90 ms, 2,93 % de errores y 2,98 % de fallos reportados por Postman. Su causa queda por analizar. Falta completar la corrida, cubrir los demás endpoints de escritura y medir desde la navegación hasta que aparecen las filas de estudiantes, usuarios e historial.

Requisito	Componente	Caso que se probó	Estado
RF-01. Iniciar sesión	Autenticación	Ingreso con credenciales válidas y rechazo de credenciales incorrectas.	Verificado.
RF-02. Registrar y consultar usuarios	Usuarios	El administrador puede crear y consultar cuentas. Se rechazan los nombres duplicados y el acceso del entrenador.	Verificado.
RF-03. Modificar y	Usuarios	Edición de cuentas, manejo de la contraseña y restricciones al eliminar usuarios.	Verificado con un hallazgo. El administrador puede

eliminar usuarios			cambiar la contraseña de otra cuenta.
RF-04. Registrar estudiantes	Estudiantes	El estudiante queda asignado al entrenador que lo registra. Se rechaza un documento repetido.	Verificado.
RF-05. Consultar estudiantes	Estudiantes	El entrenador puede consultar a sus estudiantes. El administrador puede ver el listado general.	Verificado.
RF-06. Actualizar y eliminar estudiantes	Estudiantes	El entrenador actualiza a sus estudiantes. La eliminación corresponde al administrador y se rechaza si el estudiante tiene sesiones asociadas.	Verificado.
RF-07. Abrir una sesión	Sesiones	Se abre una sesión y se rechaza una segunda apertura para el mismo estudiante. Postman 03–04.	Verificado.
RF-08. Registrar grupos musculares	Sesiones	Un grupo repetido se guarda una sola vez. Se rechazan grupos desconocidos y cambios en sesiones cerradas. Postman 06–08 y 11.	Verificado.
RF-09. Cerrar una sesión	Sesiones	Se puede cerrar una sesión con grupos registrados. Se rechaza el cierre sin grupos o si la sesión ya estaba cerrada. Postman 05, 09 y 10.	Verificado.
RF-10. Consultar el historial y eliminar sesiones	Sesiones	Se consulta el historial por estado y se comprueba que solo el administrador pueda eliminar sesiones. Postman 12 verifica el historial.	Verificado.

RNF-01. Rendimiento	API y portal	Postman con 20 usuarios virtuales en e780adf. Corrida interrumpida, con p95 global de 154 ms. Falta medir el tiempo hasta mostrar los datos de las tres listas.	Verificación parcial.
RNF-02. Seguridad	Control de acceso de la API	Se probaron solicitudes sin autorización y accesos según el rol y el estudiante asignado.	Verificado parcialmente.
RNF-03. Mantenibilidad	Código de producción medido	En la medición inicial, la complejidad más alta fue 10, dentro del límite establecido.	Cumple en la medición inicial. Falta repetirla en la versión final.

DEFINICIÓN DE MÉTRICAS

Para esta sesión elegimos cinco métricas para revisar que tan fácil era de mantener Gym Track. Algunas nos ayudaron a revisar el tamaño y la complejidad del código, otras muestran cómo se relacionan las clases y la última nos permite ver en que actividades de mantenimiento invertimos más tiempo.

Complejidad ciclomática

Campo	Descripción
Nombre de la métrica	Complejidad ciclomática V(G)
Atributo que mide	La cantidad de caminos de ejecución independientes que puede tener un método.
Fórmula o forma de cálculo	La fórmula de McCabe es V(G)= E-N +2P, donde E es el número de conexiones del grafo, N es el número de nodos y P es el número de componentes conectados. Para un solo método, normalmente P=1. En nuestro proyecto usamos el cálculo practico V(G)=1 + níumero de puntos de decisión que reconoce el medidor. Por jemplo si detecta 3 decisiones, el resultado es V(G)=1+3=4
Umbral de referencia e interpretación	Según la guía del curso, de 1 a 10 es riesgo bajo, de 11 a 20 moderado, de 31 a 50 s alto y más de 50 es muy alto. El límite definimos para Gym Track es 10 por método
Herramienta o script de medición	Usamos el medidor propio gym-track-vg-loc/1.0, con Python 3.10–3.13, Tree-sitter 0.25.2, gramática Java 0.23.5 y gramática TypeScript 0.23.2. Ejecutamos .\metricas\vg_loc\.venv\Scripts\python.exe metricas/vg_loc/measure.py --out docs/metricas/resultados/baseline/vg_loc.
Alcance de la medición	Métodos y funciones del código de producción de la API en Java y del portal en TypeScript incluidos en el análisis.
Justificación	La usamos para encontrar métodos con muchas decisiones, porque pueden requerir más casos de prueba y ser más difíciles de entender. El resultado depende de las decisiones que reconoce el medidor.
Acción asociada si se supera el umbral	Según la guía del curso, de 1 a 10 es riesgo bajo, de 11 a 20 es moderado, de 21 a 50 es alto y más de 50 es muy alto. El límite que definimos para Gym Track es de 10 por método.

Líneas de código

Campo	Descripción
Nombre de la métrica	Líneas de código, LOC.
Atributo que mide	El tamaño del código y la proporción de líneas dedicadas a comentarios.
Fórmula o forma de cálculo	LOC físicas = líneas de código + líneas de solo comentario + líneas en blanco. Las líneas efectivas incluyen las que contienen código, incluso si solo tienen una declaración o una llave. Si una línea tiene código y comentario al mismo tiempo, cuenta una sola vez, como línea de código. Por ejemplo, int total = 0; // valor inicial suma una línea de código. El porcentaje de comentarios se calcula como (líneas de solo comentario ÷ LOC físicas) × 100.
Umbral de referencia e interpretación	Según la guía del curso, más de 50 líneas efectivas por método es una señal para revisar su tamaño.
Herramienta o script de medición	Usamos el mismo medidor gym-track-vg-loc/1.0 y las mismas versiones de Python, Tree-sitter y sus gramáticas que para V(G). Ejecutamos .\metricas\vg_loc\.venv\Scripts\python.exe metricas/vg_loc/measure.py --out docs/metricas/resultados/baseline/vg_loc.
Alcance de la medición	Archivos y cuerpos de métodos del código de producción de la API y del portal incluidos en el análisis.
Justificación	Nos ayuda a encontrar métodos largos que pueden ser difíciles de leer. El resultado puede cambiar según el formato del código y la forma de clasificar las líneas.
Acción asociada si se supera el umbral	Revisar si el método hace varias tareas y separar las que se puedan entender y probar por su cuenta.

Suite CK

Campo	Descripción
Nombre de la métrica	Suite CK, de Chidamber y Kemerer.
Atributo que mide	La complejidad, la herencia, el acoplamiento, la respuesta y la cohesión de las clases.
Fórmula o forma de cálculo	WMC suma la complejidad de los métodos según CK. DIT mide la profundidad de herencia, NOC cuenta subclases directas, CBO cuenta tipos acoplados y RFC cuenta métodos que contribuyen a la respuesta de la clase. LCOM bruto compara pares de métodos que comparten atributos con los que no los comparten. LCOM* estima la cohesión mediante el uso de atributos por los métodos. La interpretación conserva las convenciones de CK 0.7.0 y distingue ambas escalas.

Umbral de referencia e interpretación	La guía indica alertas para WMC > 20, DIT > 5, CBO > 10 y RFC > 50. La referencia 0,7 se aplica a LCOM*. Para NOC, la guía no fija un límite numérico.
Herramienta o script de medición	Usamos CK 0.7.0, con Java 17 o posterior y Python 3.10–3.13. Ejecutamos .\metricas\ck_mi\.venv\Scripts\python.exe metricas/ck_mi/ck.py --out docs/metricas/resultados/baseline/ck.
Alcance de la medición	Clases públicas de Java de los paquetes service, security, controller y entity de la API.
Nombre de la métrica	Suite CK, de Chidamber y Kemerer.
Atributo que mide	La complejidad, la herencia, el acoplamiento, la respuesta y la cohesión de las clases.

Índice de mantenibilidad

Campo	Descripción
Nombre de la métrica	Índice de mantenibilidad, MI.
Atributo que mide	La mantenibilidad estimada del código a partir de su tamaño y complejidad.
Fórmula o forma de cálculo	MI bruto = 171 − 5,2 × ln(HV) − 0,23 × CC − 16,2 × ln(LOC). HV es el volumen de Halstead, calculado a partir de los operadores y operandos del código. CC es la complejidad ciclomática, LOC son las líneas efectivas y ln es el logaritmo natural. Para la escala de 0 a 100 se usa MI normalizado = máx(0, mín(100, MI bruto × 100 ÷ 171)).
Umbral de referencia e interpretación	En Gym Track usamos 65 en la escala bruta como referencia. Un valor por debajo de 65 indica que conviene revisar el código medido.
Herramienta o script de medición	Usamos el medidor propio gym-track-mi-lexical/1.0, con Python 3.10–3.13, Tree-sitter 0.25.2 y gramática Java 0.23.5. Ejecutamos .\metricas\ck_mi\.venv\Scripts\python.exe metricas/ck_mi/mi.py --vg docs/metricas/resultados/baseline/vg_loc --out docs/metricas/resultados/baseline/mi.
Alcance de la medición	Unidades de código Java de la API incluidas en el análisis.
Justificación	Nos permite revisar en un solo valor varios aspectos del código. Para comparar dos mediciones usamos la misma escala y la misma forma de contar las líneas, los operadores y los operandos.
Acción asociada si se supera el umbral	Si el MI bruto baja de 65, revisar si el método es largo o tiene muchas decisiones y evaluar cómo simplificarlo.

Esfuerzo de mantenimiento

Campo	Descripción
Nombre de la métrica	Esfuerzo de mantenimiento por tipo.
Atributo que mide	Las horas dedicadas a corregir errores, adaptar el sistema, mejorarlo y prevenir problemas.
Fórmula o forma de cálculo	Horas por tipo = suma de las horas de las actividades de ese tipo. Horas totales = horas correctivas + horas adaptativas + horas perfectivas + horas preventivas. % de esfuerzo por tipo = (horas por tipo ÷ horas totales) × 100.
Umbral de referencia e interpretación	La guía presenta esta distribución de referencia para comparar el esfuerzo: 21 % correctivo, 25 % adaptativo, 50 % perfectivo y 4 % preventivo.
Herramienta o script de medición	Usamos el medidor propio gym-track-maintenance/1.0, con Python 3.10 o posterior y Git. El script lee las horas de metricas/mantenimiento/registro.csv y consulta las etiquetas de los issues a partir de una copia guardada de GitHub. Los dos comandos se muestran abajo.
Alcance de la medición	Actividades de mantenimiento registradas durante el periodo analizado del proyecto.
Justificación	Nos permite identificar en qué tipo de trabajo usamos más tiempo. Las horas son estimadas y el costo en dinero queda sin calcular porque el equipo no registró tarifas por hora.
Acción asociada si se supera el umbral	Si un tipo concentra gran parte de las horas, revisar las actividades de ese grupo para entender qué trabajo ocupó más tiempo.

RESULTADOS

Qué revisamos	Resultado
Complejidad	En f0669d6, el valor más alto fue de 10 en el portal y de 4 en la API. Ningún método medido pasó de 50 líneas.
Acoplamiento	StudentService, StudentController y SessionService superaron la referencia de CBO de 10.
Usuarios	Juntamos la validación de nombres duplicados. La complejidad bajó de 2 a 1 al crear y de 3 a 2 al editar.
Estudiantes	Juntamos la validación de documentos duplicados. La complejidad bajó de 2 a 1 al crear y al editar.
Sesiones	Dos métodos quedaron una línea más cortos, aunque el WMC de la clase subió de 20 a 21.
Mantenimiento	Registramos 2,67 horas estimadas, con 2,50 preventivas y 0,17 correctivas.

En la línea base, StudentService obtuvo CBO 12, StudentController CBO 15 y SessionService CBO 15. Estos valores superan la referencia de 10. StudentService obtuvo LCOM* de 0,8125, una señal para revisar la cohesión. El MI bruto mínimo observado fue aproximadamente 86,02 en una unidad de SecurityConfig, por encima de la referencia de 65 en el alcance medido. De las 2,67 horas estimadas registradas, el 93,63 % correspondió a mantenimiento preventivo y el 6,37 % a correctivo. Estos porcentajes describen únicamente el esfuerzo registrado.

Figura 2: Acoplamiento por clase

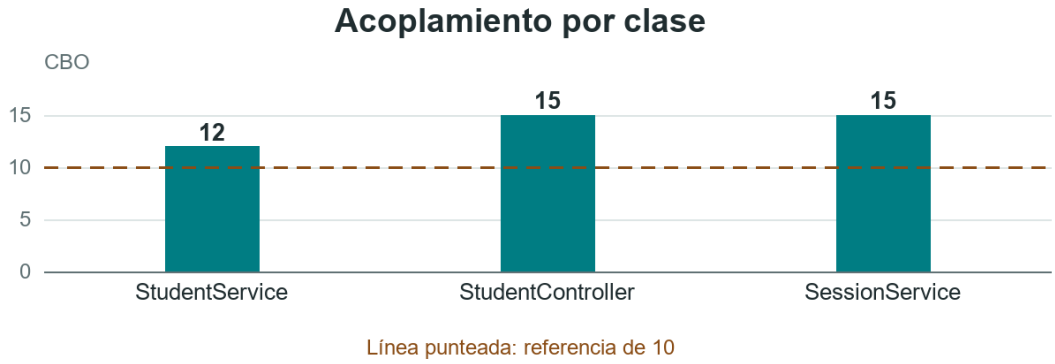
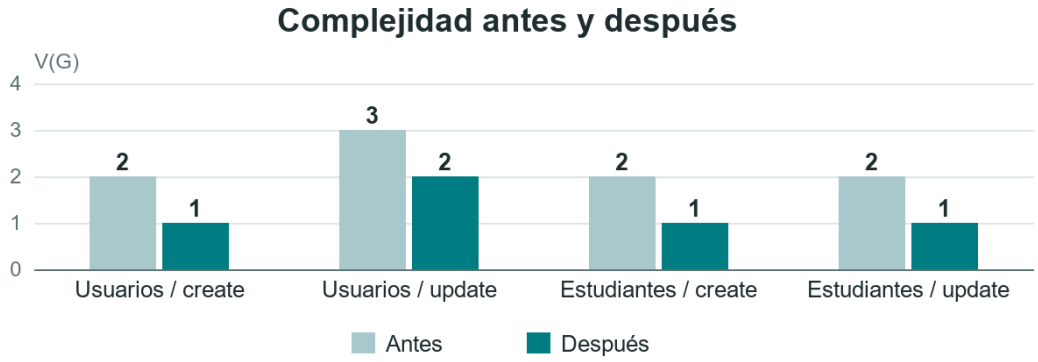


Figura 3: Complejidad antes y después



HALLAZGOS Y MEJORAS

Prioridad	Qué encontramos	Qué falta hacer
Alta	Un administrador puede cambiar la contraseña de otra cuenta.	Separar el cambio de contraseña de la edición del perfil y probar que solo el titular pueda hacerlo.
Alta	Rendimiento parcialmente verificado. Postman detuvo la prueba local por un límite de recursos.	Completar la prueba de escritura con 20 usuarios virtuales, analizar los fallos del alta y medir por separado el tiempo hasta mostrar cada lista.
Alta	Las pruebas de permisos cubren solo algunos casos.	Completar las pruebas por rol y probar con token inválido o sin token.
Media	El portal reportó 16 alertas moderadas en sus dependencias.	Revisar qué paquetes están afectados y cuáles se pueden actualizar.

Media	Tres clases tienen un acoplamiento alto.	Revisar qué dependencias necesita cada clase y si se pueden separar responsabilidades.
Por confirmar	La app tiene un historial por estudiante.	Preguntarle al docente si esa vista cumple el requisito de reportes.

PLAN DE MEJORA

Nuestra primera prioridad es revisar cómo se cambian las contraseñas. Confirmamos que un administrador puede cambiar la contraseña de otra cuenta desde la edición del perfil, sin que participe el titular. Proponemos separar esas dos operaciones y pedir al titular que confirme su contraseña actual para cambiarla. Para comprobar la mejora, probaríamos que el administrador pueda seguir editando el perfil sin cambiar la contraseña y que un intento de hacerlo deje la contraseña anterior intacta. Este cambio sigue pendiente.

También necesitamos completar la verificación de rendimiento. En la prueba local de Postman sobre `e780adf`, con 20 usuarios virtuales configurados, registramos 7.900 solicitudes y un p95 global de 154 ms antes de que la corrida se detuviera por un límite de recursos del equipo. Los tiempos de las operaciones probadas quedaron por debajo de 500 ms, pero el alta de estudiantes registró errores y fallos cuya causa no hemos confirmado. El siguiente paso es revisar esos casos, repetir la corrida hasta completarla, medir los endpoints de escritura que faltan y comprobar cuánto tarda cada lista en mostrar sus datos.

En seguridad, las pruebas que ejecutamos cubren algunos casos de acceso por rol. Queda pendiente revisar las rutas protegidas y completar los casos con token ausente o inválido y con roles sin permiso. En el portal, la instalación reportó 16 alertas moderadas en las dependencias. Primero

revisaríamos qué paquetes están afectados y cómo los usa el proyecto para decidir si corresponde actualizarlos.

En mantenibilidad, revisaríamos las dependencias de las clases que superaron la referencia de acoplamiento antes de decidir si conviene separar responsabilidades. Las comparaciones mostraron que simplificar un método no siempre mejora la métrica de toda la clase. En sesiones, por ejemplo, dos métodos quedaron más cortos, pero el WMC subió de 20 a 21. Si aplicamos otro cambio, repetiríamos las mismas mediciones y las pruebas del módulo para comparar el resultado. Por separado, queda pendiente confirmar con el docente si el historial por estudiante cubre el requisito de reportes.

Para ordenar el trabajo, estimamos que revisar las alertas de dependencias requiere menos esfuerzo que cambiar el flujo de contraseñas. Completar las pruebas de autorización y rendimiento requiere preparar casos, datos y un entorno donde podamos terminar la corrida. Son estimaciones cualitativas para planear el trabajo futuro. Las horas registradas en el Anexo C corresponden al trabajo que ya realizamos.

APORTES INDIVIDUALES

Juan Camilo Bernal Gordillo: Preparé las pruebas de autenticación y gestión de usuarios, medí la complejidad ciclomática y las líneas de código, y centralicé la validación de nombres de usuario duplicados. También actualicé el README con las instrucciones de ejecución y los resultados de calidad. Ejecuté la prueba local de rendimiento con Postman sobre e780adf, con 20 usuarios virtuales. La corrida registró 7.900 solicitudes y un p95 global de 154 ms antes de detenerse por un límite de recursos del equipo. Las capturas respaldan la verificación parcial del RNF-01. Evidencia: PR #17, #34 y #35, y capturas de Postman.

Laura Marcela Quintero Poveda: Desarrolle las pruebas de estudiantes y centralicé la validación de documentos duplicados. Apliqué la suite CK y el índice de mantenibilidad, e interpreté los resultados de acoplamiento y cohesión. También verifiqué la compilación y las pruebas del portal sobre fa80144, y documenté los requisitos, la trazabilidad y la arquitectura del proyecto. Evidencia: PR #30 y #32, y commit c22c9ed.

Cristian Andrés Muñoz Montenegro: Consolidé las pruebas de sesiones y agrupé las comprobaciones de pertenencia y apertura en el servicio de sesiones. Registré el esfuerzo de mantenimiento y consolidé las tablas y gráficos del Anexo C. También verifiqué el arranque integrado con Docker, el inicio de sesión y la colección funcional de Postman sobre fa80144, que registró 12 peticiones y 24 aserciones aprobadas, con 0 fallos. Evidencia: PR #28, #31 y #33, y resultados de la verificación integrada.

DECLARACIÓN DE USO DE IA

Reutilizamos la aplicación base de Gym Track de la asignatura Desarrollo Fullstack. Para este informe usamos ChatGPT y Codex como apoyo para redactar y revisar la documentación, preparar los scripts de medición y organizar el análisis. Los valores que reportamos provienen de las pruebas y mediciones del equipo. En el informe indicamos qué resultados pudimos comprobar y cuáles quedaron pendientes.

CONCLUSIONES

Gym Track funcionó de forma integrada y pasó las pruebas reportadas en el commit fa80144. En usuarios y estudiantes, reunimos las validaciones de duplicados en un solo lugar y redujimos la complejidad de los métodos de crear y editar. En sesiones, dos métodos quedaron más cortos, aunque el WMC de la clase subió de 20 a 21. Esto nos ayudó a ver que un cambio puede simplificar un método y, al mismo tiempo, aumentar la complejidad medida en la clase.

La revisión detectó alertas de acoplamiento en la API. La prioridad de seguridad es corregir el cambio de contraseña de otra cuenta por un administrador y comprobar que el titular controle el cambio. La autorización sigue parcialmente verificada. En rendimiento, el p95 observado fue favorable para las operaciones probadas, pero la interrupción y la cobertura parcial impiden acreditar el RNF-01 completo

REFERENCIAS

Zakaria, F., & Utami, E. (2024). An analysis of requirement engineering and techniques: A literature review. International Journal of Research and Applied Technology, 4(1). https://doi.org/10.34010/injuratech.v4i1.13731

Dongmo, C. (2024). A review of non-functional requirements analysis throughout the SDLC. Computers, 13(12), Article 308. https://doi.org/10.3390/computers13120308

Kim, M., Xin, Q., Sinha, S., & Orso, A. (2022). Automated test generation for REST APIs: No time to rest yet. En Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2022). ACM.

Corporación Universitaria del Huila (CORHUILA). *Guía del proyecto final de Métricas y Estándares de Calidad*. Docente José Miguel Llanos Mosquera.

Equipo Gym Track. (2026). Gym Track: repositorio del proyecto final de Métricas y Estándares de Calidad. https://github.com/laura-marcela05/proyecto-final